

UNIVERSIDADE FEDERAL DE SANTA CATARINA

Projeto Hierárquico

EEL5105 – Circuitos e Técnicas Digitais

Objetivos

- Entender o conceito de **Projeto Hierárquico**.
- Implementar **projetos** em **VHDL** usando **component** e **port map**.

Introdução ao Projeto Hierárquico

Tarefas

Tarefa Avançada

Introdução ao Projeto Hierárquico

- **Projeto Hierárquico**
  - Abordagem de projeto usada não somente em **VHDL**.
  - **Idéia**: estruturar o projeto em múltiplos **componentes** que podem ser **criados separadamente** e depois **integrados** e **reutilizados**.
  - Facilita a **leitura**, **entendimento** e **manutenção** do código.

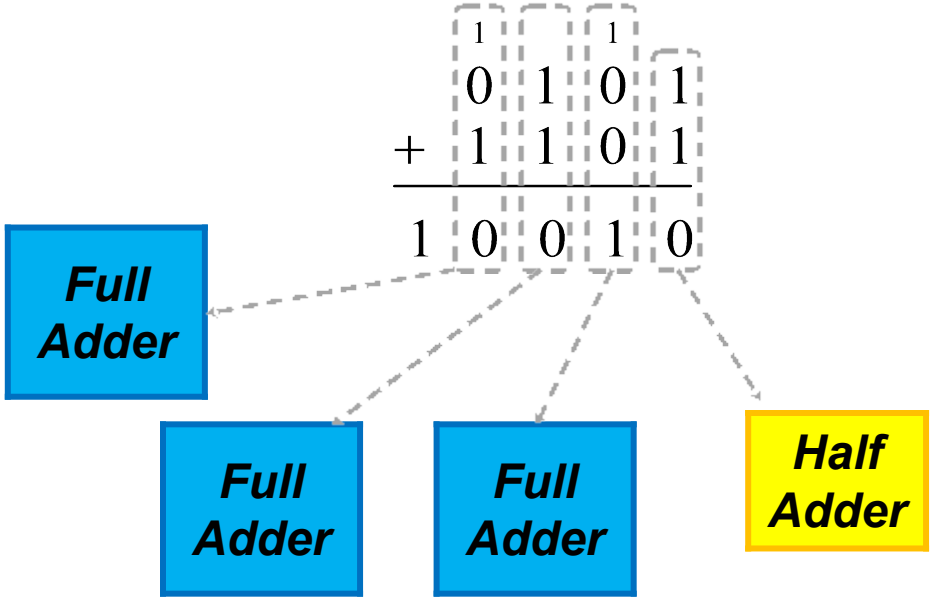
Introdução

- Existem diferentes modelos para declarar a lógica a ser configurada no FPGA:

  - ***Dataflow*** **(circuitos combinacionais - aula anteriores)**
  - ***Structural*** **(esta aula)**
  - ***Behavioral*** **(utilizada em circuitos sequenciais)**
  - ***Mix*** **(combinação dos modelos anteriores)**

Introdução

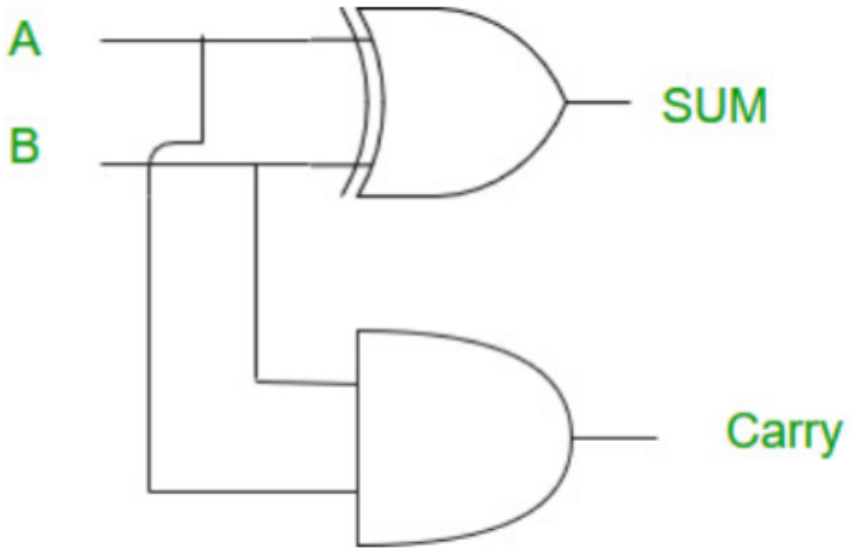
- **Exemplo 1**: somador de números de **4 bits** construído usando um ***half adder*** e ***full adders***



Introdução

Half adder

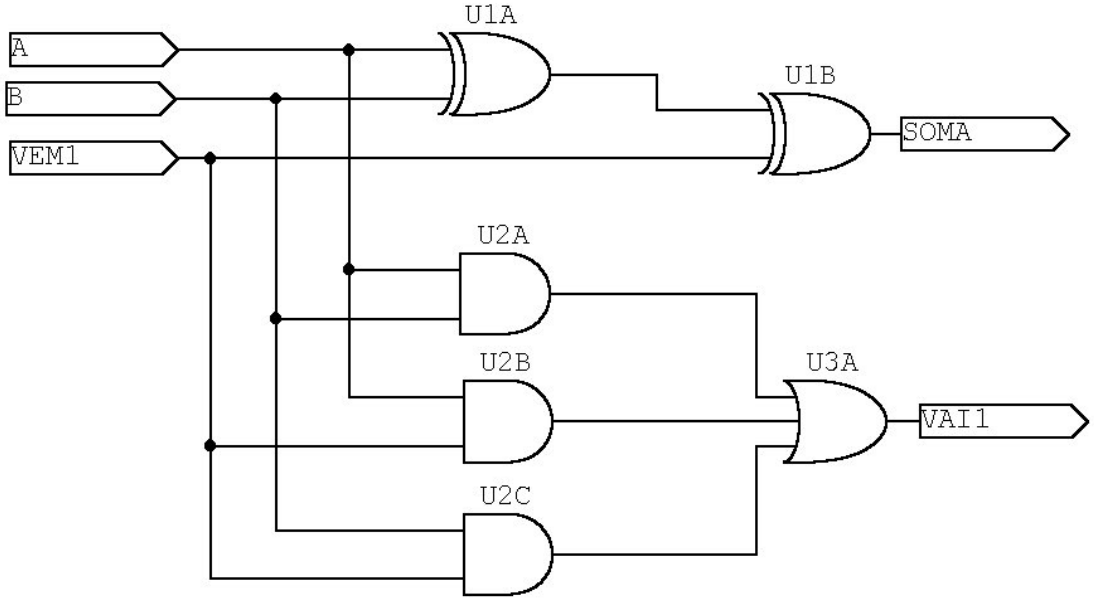
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Sum <= (A xor B);
Carry <= (A and B);

Introdução

Full adder

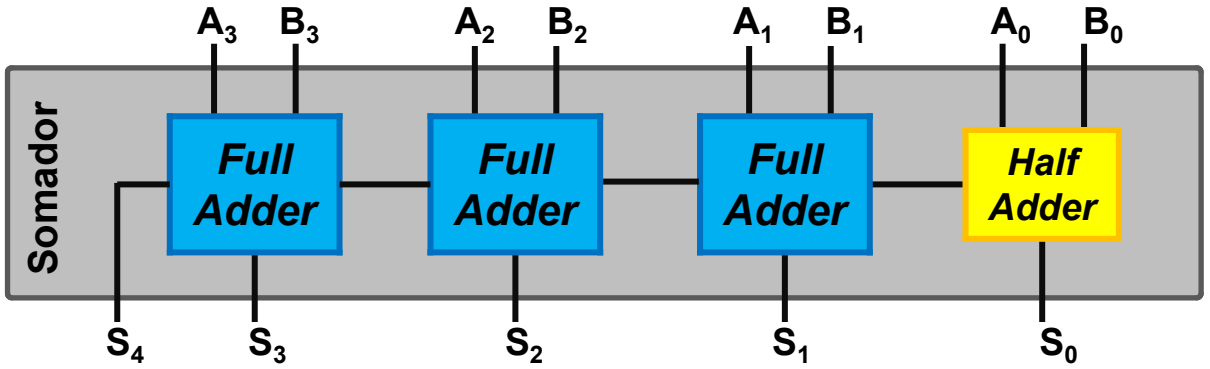


SOMA <= (A xor B) xor VEM1;
VAI1 <= (A and B) or (B and VEM1) or (A and VEM1);

Truth Table				
C-in	B	A	Sum	C-out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

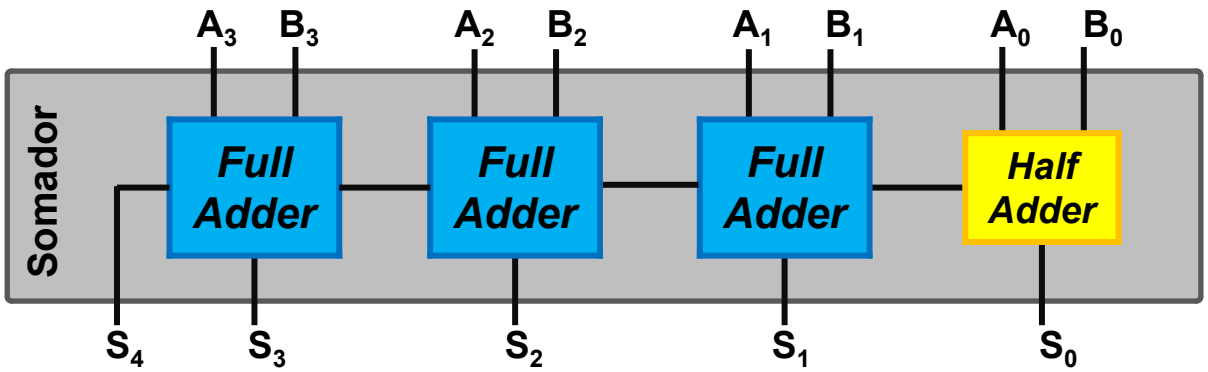
Introdução

- **Exemplo 1**: somador de números de **4 bits** construído usando um ***half adder*** e ***full adders***
  - **Componentes** internos são primeiramente projetados: **Full Adder** **Half Adder**
  - Em seguida, são **integrados** para construir o somador desejado:



Introdução

- **Exemplo 1**: somador de números de **4 bits** construído usando um ***half adder*** e ***full adders***
  - **Circuito**:

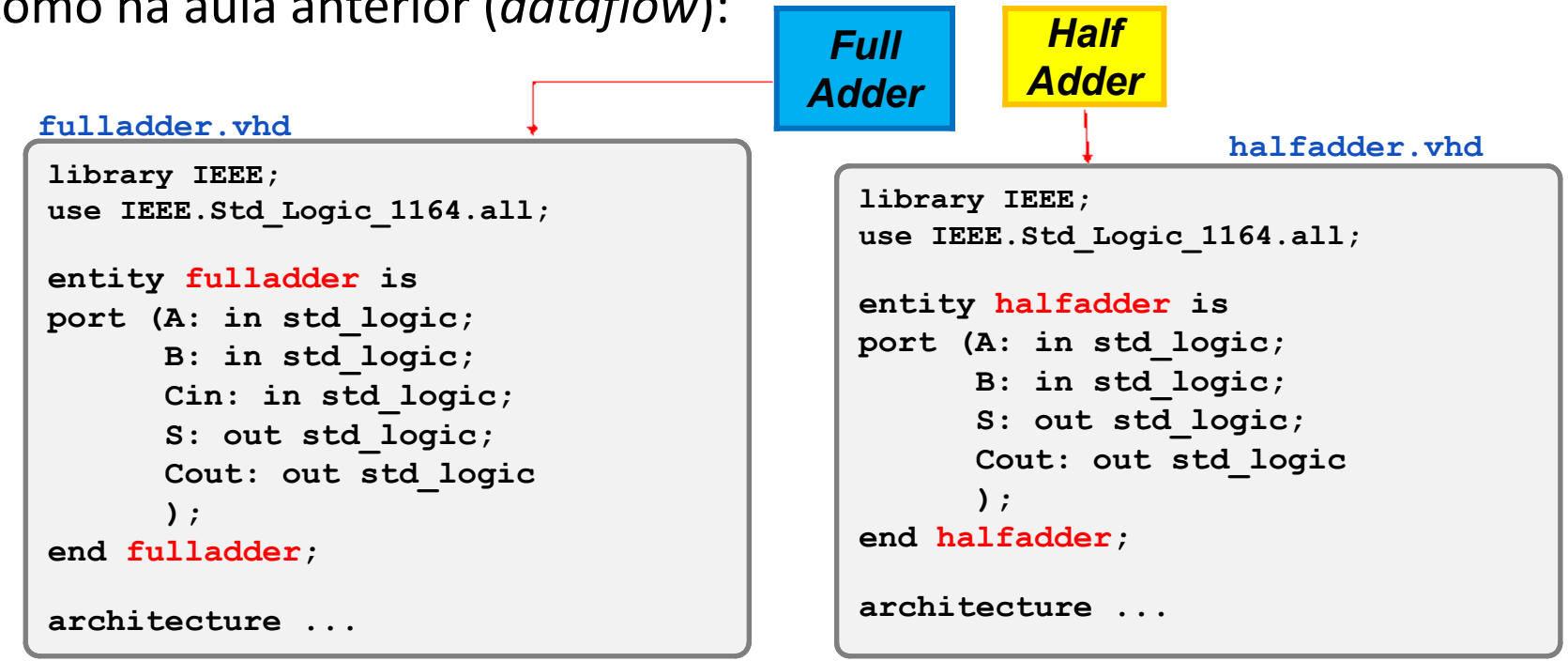


- **Hierarquia**:

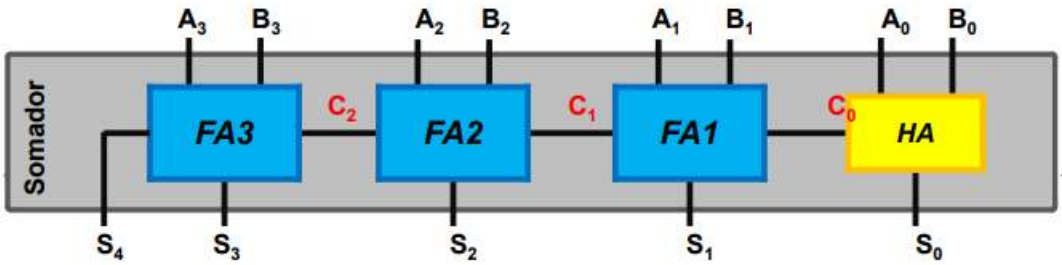
somador.vhd
- fulladder.vhd
- halfadder.vhd

Introdução

- **Exemplo 1**: Componentes integrantes são primeiramente projetados como na aula anterior (*dataflow*):



Introdução



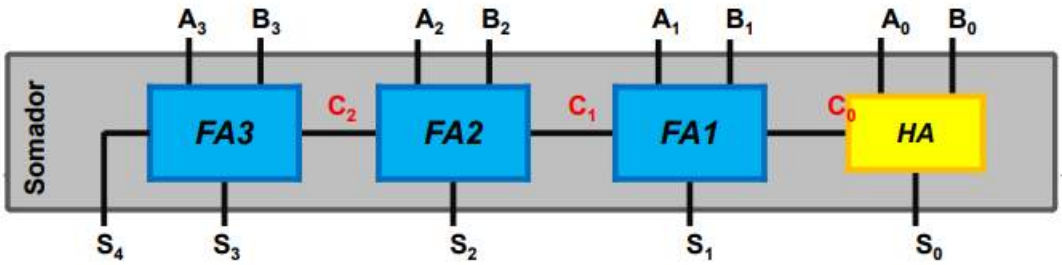
- **Exemplo 1:**

```
library IEEE;
use IEEE.Std_Logic_1164.all;
entity somador is
port (A,B: in std_logic_vector(3 downto 0);
      S: out std_logic_vector(4 downto 0));
end somador;
architecture soma4 of somador is
 signal C0,C1,C2: std_logic;
 component halfadder is
  port (A: in std_logic;
        B: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;
 component fulladder is
  port (A: in std_logic;
        B: in std_logic;
        Cin: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;
```

Associação Declarada

```
begin
 HA: halfadder port map (A => A(0),
                         B => B(0),
                         S => S(0),
                         Cout => C0);
 FA1: fulladder port map (A => A(1),
                          B => B(1),
                          Cin => C0,
                          S => S(1),
                          Cout => C1);
 FA2: fulladder port map (A => A(2),
                          B => B(2),
                          Cin => C1,
                          S => S(2),
                          Cout => C2);
 FA3: fulladder port map (A => A(3),
                          B => B(3),
                          Cin => C2,
                          S => S(3),
                          Cout => S(4));
end soma4;
```

Introdução



- **Exemplo 1:**

```
library IEEE;
use IEEE.Std_Logic_1164.all;
entity somador is
port (A,B: in std_logic_vector(3 downto 0);
      S: out std_logic_vector(4 downto 0));
end somador;
architecture soma4 of somador is
 signal C0,C1,C2: std_logic;

 component halfadder is
  port (A: in std_logic;
        B: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;

 component fulladder is
  port (A: in std_logic;
        B: in std_logic;
        Cin: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;
```

```
begin
 HA: halfadder port map (A => A(0),
                         B => B(0),
                         S => S(0),
                         Cout => C0);

 FA1: fulladder port map (A => A(1),
                          B => B(1),
                          Cin => C0,
                          S => S(1),
                          Cout => C1);

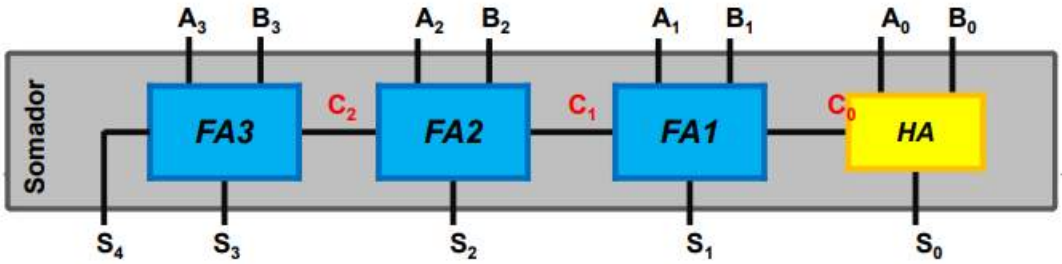
 FA2: fulladder port map (A => A(2),
                          B => B(2),
                          Cin => C1,
                          S => S(2),
                          Cout => C2);

 FA3: fulladder port map (A => A(3),
                          B => B(3),
                          Cin => C2,
                          S => S(3),
                          Cout => S(4));

end soma4;
```

Declarações dos componentes já existentes (de outros arquivos)

Introdução



- **Exemplo 1**:

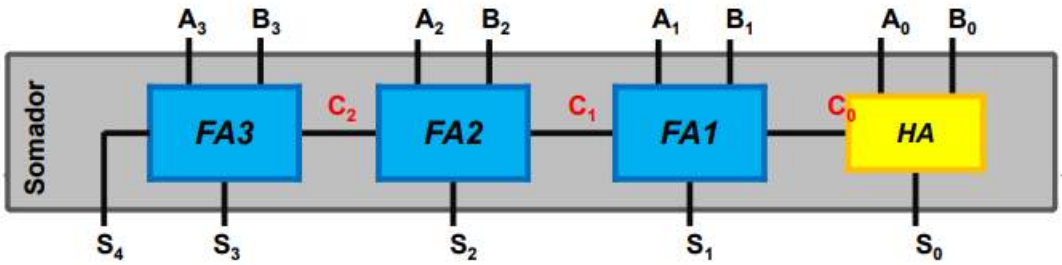
```
library IEEE;
use IEEE.Std_Logic_1164.all;
entity somador is
port (A,B: in std_logic_vector(3 downto 0);
      S: out std_logic_vector(4 downto 0));
end somador;
architecture
 signal C0,C
 component ha
  port (A: i
        B: i
        S: o
        Cout
 end compone
 component fulladder is
  port (A: in std_logic;
        B: in std_logic;
        Cin: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;
```

Descrição das conexões dos componentes (note que múltiplas instâncias de um mesmo componente podem ser utilizadas)

```
begin
HA: halfadder port map (A => A(0),
                        B => B(0),
                        S => S(0),
                        Cout => C0);
FA1: fulladder port map (A => A(1),
                         B => B(1),
                         Cin => C0,
                         S => S(1),
                         Cout => C1);
FA2: fulladder port map (A => A(2),
                         B => B(2),
                         Cin => C1,
                         S => S(2),
                         Cout => C2);
FA3: fulladder port map (A => A(3),
                         B => B(3),
                         Cin => C2,
                         S => S(3),
                         Cout => S(4));

end soma4;
```

Introdução



- **Exemplo 1**
  (forma alternativa para **port map** / associação por posição):

```
library IEEE;
use IEEE.Std_Logic_1164.all;

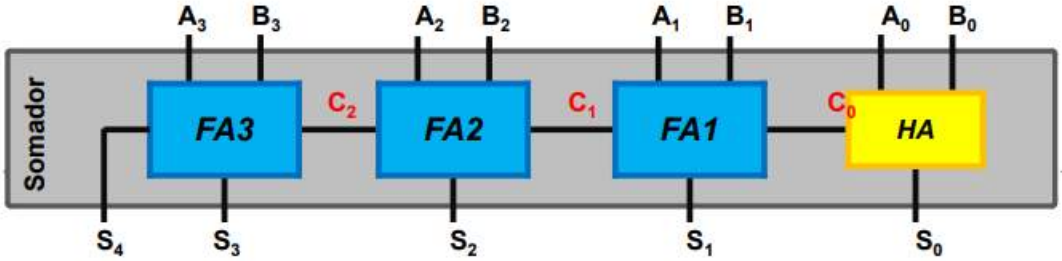
entity somador is
port (A,B: in std_logic_vector(3 downto 0);
      S: out std_logic_vector(4 downto 0)
     );
end somador;

architecture soma4 of somador is
 signal C: std_logic_vector(3 downto 0);
 component halfadder is
  port (A: in std_logic;
        B: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;
```

```
component fulladder is
  port (A: in std_logic;
        B: in std_logic;
        Cin: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;

begin
 HA: halfadder port map (A(0),B(0),S(0),C(0));
 FA1: fulladder port map (A(1),B(1),C(0),S(1),C(1));
 FA2: fulladder port map (A(2),B(2),C(1),S(2),C(2));
 FA3: fulladder port map (A(3),B(3),C(2),S(3),S(4));
end soma4;
```

Introdução



- **Exemplo 1**
  (forma alternativa para **port map** / associação por posição):

```
library IEEE;
use IEEE.Std_Logic_1164.all;

entity som
port (A,B
      S
);
end somador;

architecture soma4 of somador is
 signal C0,C1,C2: std_logic;

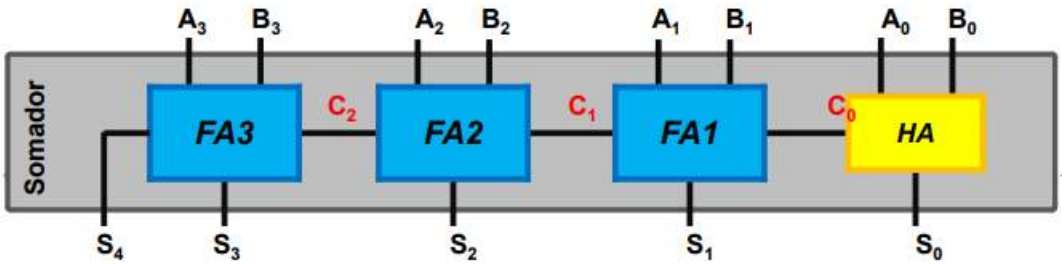
component halfadder is
 port (A: in std_logic;
       B: in std_logic;
       S: out std_logic;
       Cout: out std_logic);
end component;
```

Declarações dos componentes já existentes (de outros arquivos)

```
component fulladder is
  port (A: in std_logic;
        B: in std_logic;
        Cin: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
end component;

begin
 HA: halfadder port map (A(0),B(0),S(0),C(0));
 FA1: fulladder port map (A(1),B(1),C(0),S(1),C(1));
 FA2: fulladder port map (A(2),B(2),C(1),S(2),C(2));
 FA3: fulladder port map (A(3),B(3),C(2),S(3),S(4));
end soma4;
```

Introdução



- **Exemplo 1**
  (forma alternativa para **port map** / associação por posição):

```
library IEEE;
use IEEE.Std_Logic_1164.all;

entity somador is
port (A,B: in std_logic_vector(3 downto 0);
      S: out                4 downto 0)
     );
end somador;

architecture soma4 of somador is

 signal C0,C1,C2: std_logic;

 component halfadder is
  port (A: in std_logic;
        B: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;
```

Descrição das conexões dos componentes

```
component fulladder is
  port (A: in std_logic;
        B: in std_logic;
        Cin: in std_logic;
        S: out std_logic;
        Cout: out std_logic);
 end component;

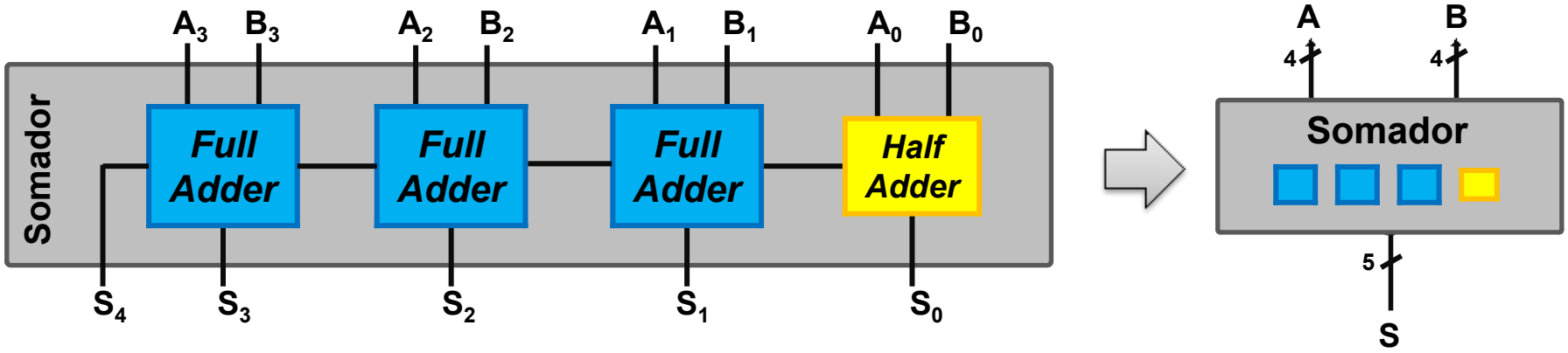
begin

HA: halfadder port map (A(0),B(0),S(0),C(0));
FA1: fulladder port map A(1),B(1),C(0),S(1),C(1);
FA2: fulladder port map (A(2),B(2),C(1),S(2),C(2));
FA3: fulladder port map (A(3),B(3),C(2),S(3),S(4));

end soma4;
```

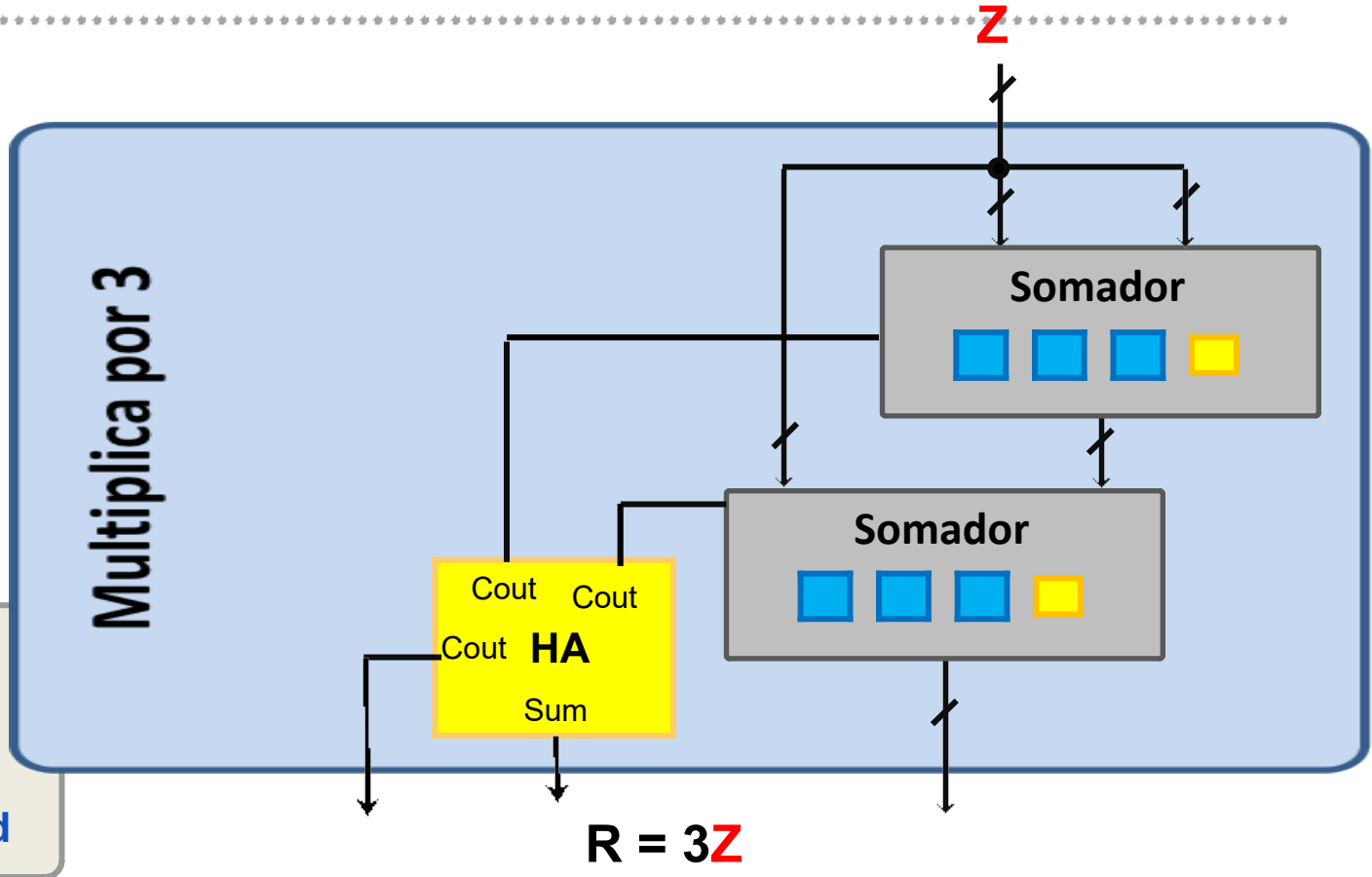
Introdução

- **Exemplo 2**: multiplicador por 3 construído usando somadores, que por sua vez foram construídos com ***half adder*** e ***full adders***



Introdução

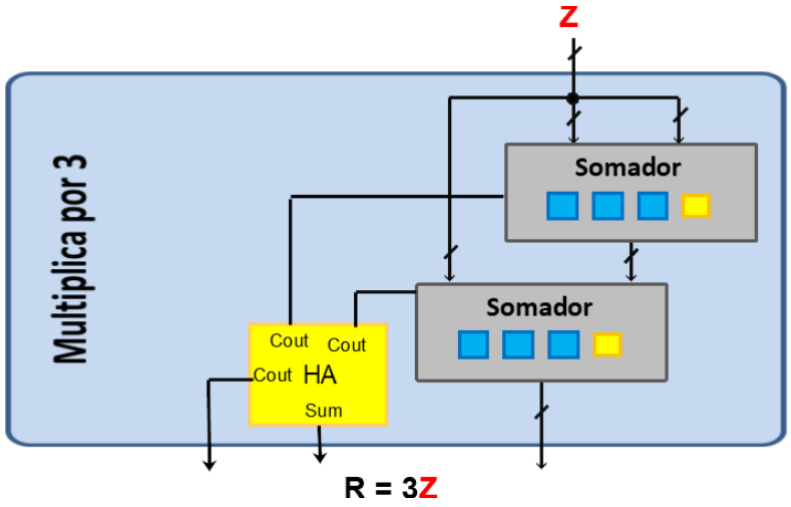
- **Exemplo 2**: multiplicador por 3 construído usando somadores, que por sua vez foram construídos com *half adder* e *full adders*.



mult3.vhd
- somador.vhd
  - fulladder.vhd
  - halfadder.vhd

Introdução

- **Exemplo 2**:




```
library IEEE;
use IEEE.Std_Logic_1164.all;

entity mult3 is
port (Z: in std_logic_vector(3 downto 0);
      Y: out std_logic_vector(5 downto 0) );
end mult3;

architecture mult3arch of mult3 is

 signal W,V: std_logic_vector(4 downto 0);

 component somador is
  port (A,B: in std_logic_vector(3 downto 0);
        S: out std_logic_vector(4 downto 0) );
 end component;
component halfadder is
          port (    A: in std_logic;
                    B: in std_logic;
                    Sum: out std_logic;
                    Cout: out std_logic);
end component;
begin

SUM1: somador port map (Z,Z,W);

SUM2: somador port map (W(3 downto 0),Z,V);

HA:   halfadder port map (W(4),V(4),Y(4),Y(5));

Y(3 downto 0) <= V(3 downto 0);
end mult3arch;
```

Introdução ao Projeto Hierárquico

Tarefas

Tarefa Avançada

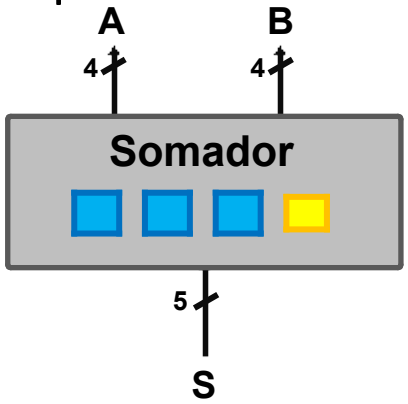
Tarefas

Observação importante: mapeamento de **ports** para **SW/LEDR/KEY** deve ser realizado apenas para a **top level entity** (topo de hierarquia do projeto). Ou seja, é necessário:

- Nomear as **ports** da **top level entity** como **SW/LEDR/KEY**
- No **emulador**, pode-se conectar **ports** da **top level entity** a **SW/LEDR/KEY** usando o **Mapper**;

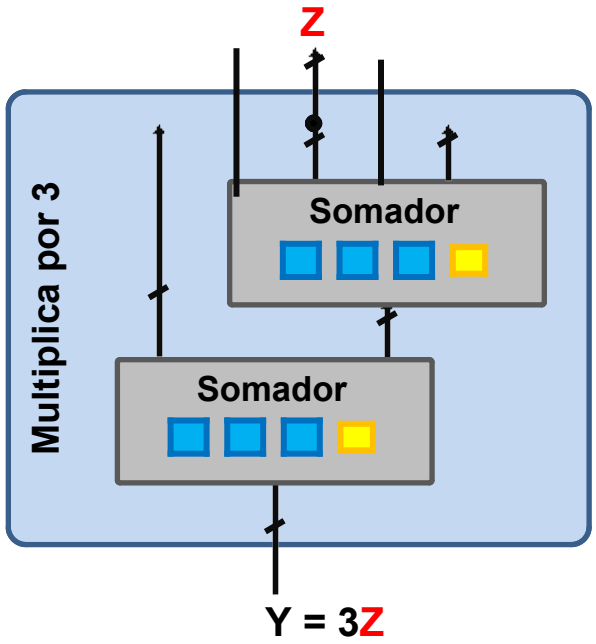
Tarefa 1

- Implementar o **Somador de 4 Bits** mostrado anteriormente no kit DE2. Para tal:
  - Faça a implementação do **half adder** no arquivo **halfadder.vhd**.
  - Similarmente, faça a implementação do **full adder** no arquivo **fulladder.vhd**.
  - Faça a implementação do **somador de 4 bits** no arquivo **somador.vhd**.
  - Finalmente, faça o mapeamento de portas e o execute no kit DE2.



Tarefa 2

- Implementar no kit DE2, o **Multiplicador por 3** discutido anteriormente seguindo passos similares aos usados para implementar o **Somador de 4 Bits**.

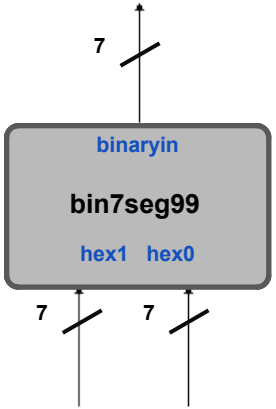


Tarefa 3

- Implementar um **Somador de 4 Bits** com entradas e saídas apresentadas em displays de 7 segmentos, usando o componente **bin7seg** disponível no Moodle.

  - **bin7seg**: recebe entradas de 5 bits e converte o valor binário dessas entradas para displays de 7 segmentos, funcionando para valores de entrada de **0** a **1F**.

```
entity bin7seg is
    port (
        binaryin: in std_logic_vector (4 downto 0);
        OUT1, OUT0: out std_logic_vector (6 downto 0)
    );
end bin7seg;
```

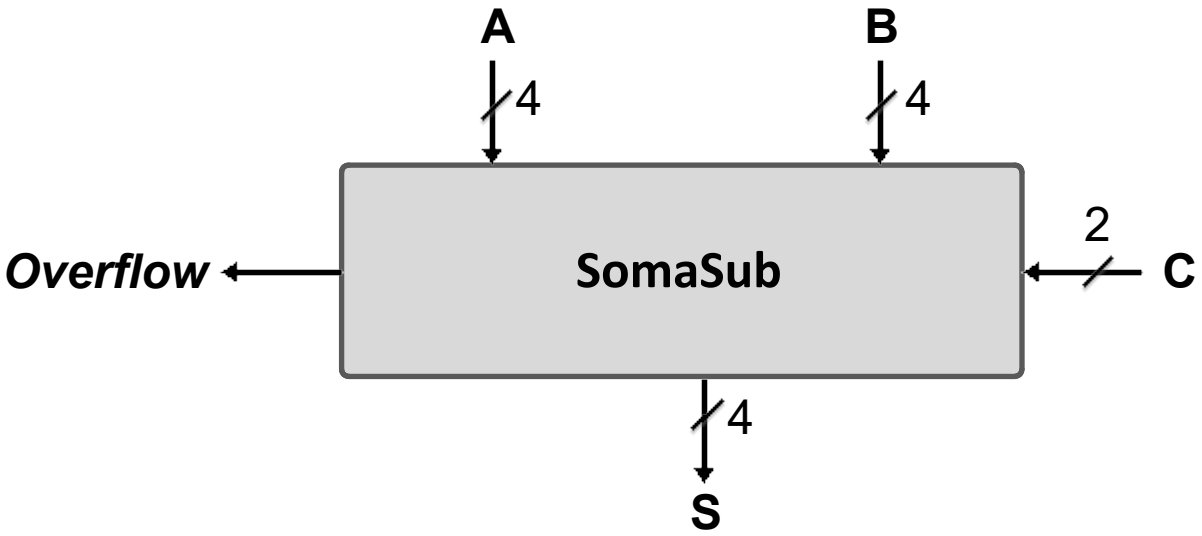


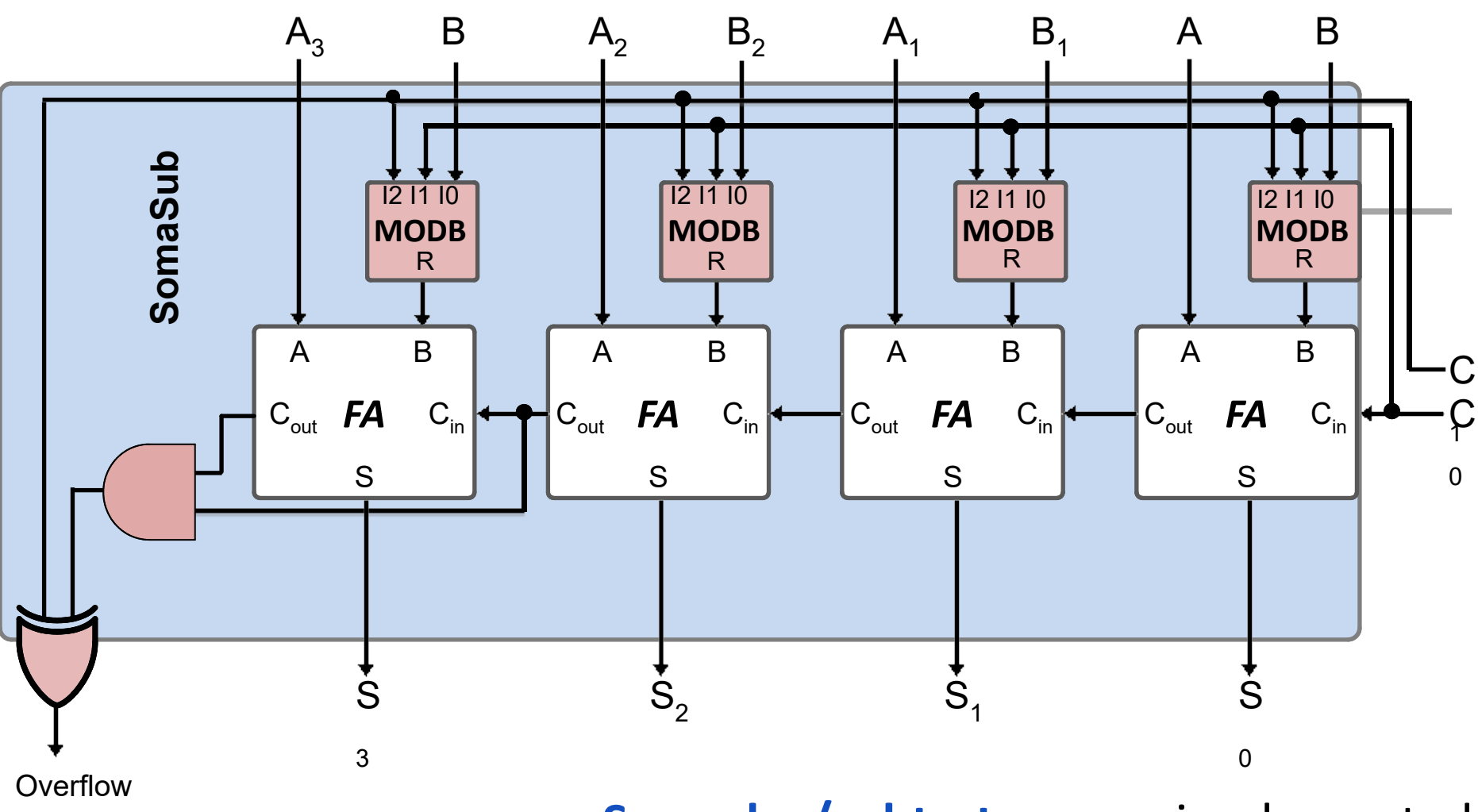
Introdução ao Projeto Hierárquico
Tarefas

Tarefa Avançada

- Implementar um **circuito somador/subtrator de 4 bits** capaz de realizar **soma** ou **subtração** com dois operandos, ou ainda **incremento** e **decremento** de um dos operandos.

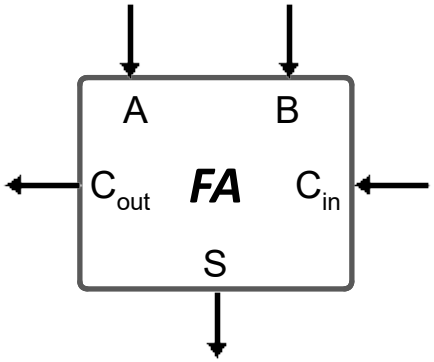
C	Operação
0 0	A + B
0 1	A + 1
1 0	A – 1
1 1	A – B



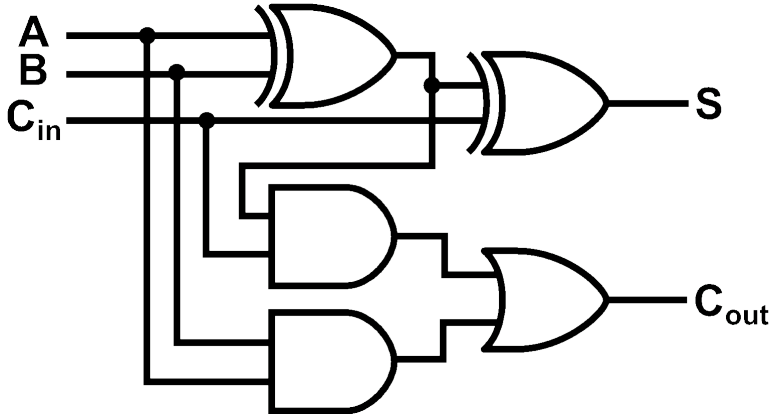


- **Somador/subtrator** a ser implementado

- Implementação do **Somador/subtrator**
  - Primeiramente, implementar **componentes** básicos (*full-adder*) :

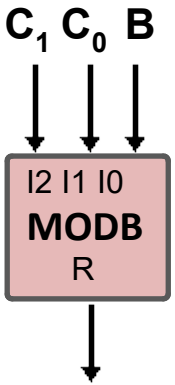


A B C_{in}	C_{out}	S
0 0 0	0	0
0 0 1	0	1
0 1 0	0	1
0 1 1	1	0
1 0 0	0	1
1 0 1	1	0
1 1 0	1	0
1 1 1	1	1

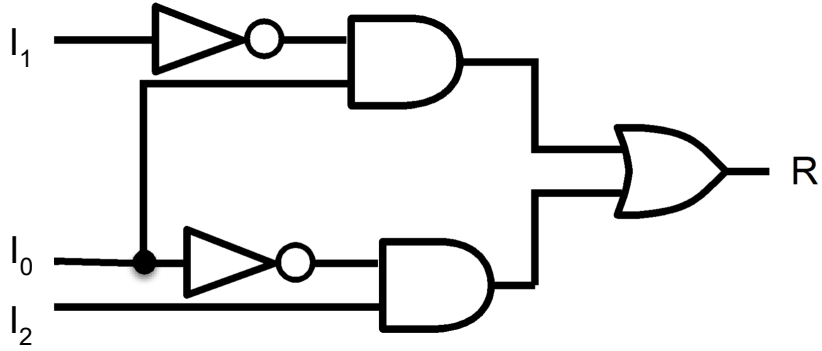


Outra maneira de implementar o somador completo

- Implementação do **Somador/subtrator**
  - Primeiramente, implementar **componentes** básicos (controle)
  - (OBS: números negativos em complemento de 2):



$C_1 C_0$ B	R	
0 0 0	0	A+B
0 0 1	1	
0 1 0	0	A+1
0 1 1	0	
1 0 0	1	A-1
1 0 1	1	
1 1 0	1	A-B
1 1 1	0	

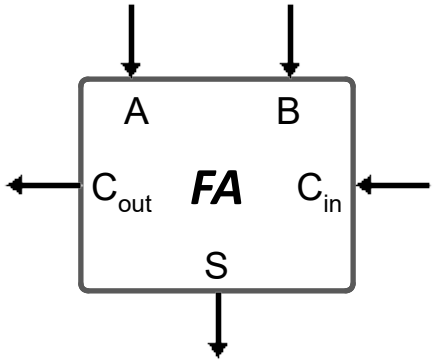


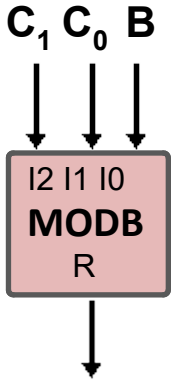
C_0 é o *carry in* de S_0 (slide 28)

Faz B = 1111 (-1 em decimal)

Faz B = /B + C_0 (igual a -B)

- Implementação do **Somador/subtrator**
  - Primeiramente, implementar **componentes** básicos:





somasub.vhd

- Em seguida, integrá-los em um arquivo topo, denominado **somasub.vhd**:

fulladder.vhd

modb.vhd

Tarefa

- **Passo 1**: Criar projeto no **Quartus II**
  - Acessar **File -> New Project Wizard** e criar um projeto como feito na aula anterior.
    - Sugestão de nome do projeto: **somasub**

- **Passo 2**: Criar arquivos do tipo **VHDL** com a descrição dos componentes internos (**fulladder.vhd** e **modb.vhd**).

- **File -> New -> Design Files -> VHDL File**
- **File -> Save As** com nomes desejados
- **Atenção:** nomes dos **arquivos vhdl** devem ser iguais aos das suas respectivas **entities**

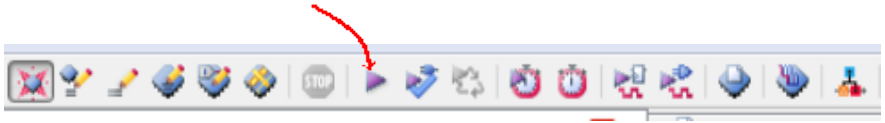
Tarefa

- **Passo 3**: Criar outro arquivo **VHDL** com a descrição componente topo (**somasub.vhd**).
- **File -> New -> Design Files -> VHDL File**
- **File -> Save As** como **somasub.vhd**.
- Com **somasub.vhd** aberto: **Project -> Set As Top Level Entity**
- Utilize o seguinte código como descrição da entity **somasub**:

```
entity somasub is
port (A: in std_logic_vector(3 downto 0);
      B: in std_logic_vector(3 downto 0);
      C: in std_logic_vector(1 downto 0);
      S: out std_logic_vector(3 downto 0);
      Overflow: out std_logic
      );
end somasub;
```

Tarefa

- **Passo 4**: Faça a **síntese** do circuito a partir de **Processing -> Start Compilation**



- **Passo 5**: Uma vez compilado e verificado que não tem erros de sintaxe **simular** o circuito no ***ModelSim*** da seguinte forma.

  a) A=2 e B=7 para as 4 operações possíveis (A+B, A-1, A+1 e A-B).
  b) A=-5 e B=7 para as 4 operações possíveis (A+B, A-1, A+1 e A-B).

- **Passo 6:** Visualize o diagrama de blocos gerado **Tools-> Netlist Viewers-> RTL Viewer.**